In [1]:
```
import sys
sys.executable
```

Out[1]:
```
'c:\\kalpesh\\1.ITvedant\\2.ML projects\\Silage_Sales_ML_Project\\venv\\Scripts\\python.exe'
```

In [2]:
```
import pandas as pd
from sqlalchemy import create_engine
```

In [3]:
```
username = "root"
password = "root"
host = "localhost"
port = "3306"
database = "Pancham_Silage_Factory_DB"

engine = create_engine(f"mysql+pymysql://{username}:{password}@{host}:{port}/{database}")
```

In [4]:
```
engine.connect()
```

Out[4]:
```
<sqlalchemy.engine.base.Connection at 0x20da257a650>
```

In [5]:
```
version = sys.version
print(version)
```

```
3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
```

Smart Silage Pricing – Business EDA

Agricultural Supply Chain Analytics

This notebook analyzes silage crop trading transactions to understand pricing dynamics, logistics costs, customer demand patterns, and overall profitability performance across the agricultural supply chain.

Objectives

- Identify high-revenue crop types
- Understand seasonal pricing trends
- Evaluate customer demand patterns
- Analyze key profitability drivers
- Study the impact of logistics costs
- Identify payment behavior and assess credit risk

1. Importing essential python libraries

In [6]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings('ignore')
```

2. Dataset Loading

In [7]:
```
df = pd.read_sql("SELECT * FROM main_data", engine)
df.head()
```

Out[7]:

	Transaction_ID	Date	Customer_Type	Crop_Type	Harvest_Season	Moisture_Content_Percent	DM_Content_Percent	Quantity_M
0	TXN2022-000077	2022-07-08	Co-operative	Maize	Kharif	69.1	30.9	9.3
1	TXN2022-000092	2022-09-28	Individual Farmer	Bajra	Kharif	67.4	32.6	8.2
2	TXN2022-000126	2022-12-06	Individual Farmer	Maize	Rabi	64.8	35.2	12.1
3	TXN2022-000128	2022-12-06	Dairy Farm	Bajra	Rabi	59.1	40.9	3.4
4	TXN2022-000133	2022-06-11	Individual Farmer	Jowar	Summer	68.4	31.6	5.6

3. Data Cleaning & Preprocessing

In [8]:

```
df.info()
df.isnull().sum()
df.columns = df.columns.str.strip()
df.columns = df.columns.str.title()
df.fillna('0',inplace=True)
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4759 entries, 0 to 4758
Data columns (total 15 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Transaction_ID            4759 non-null   object
 1   Date                      4759 non-null   object
 2   Customer_Type             4759 non-null   object
 3   Crop_Type                 4759 non-null   object
 4   Harvest_Season            4759 non-null   object
 5   Moisture_Content_Percent  4759 non-null   float64
 6   DM_Content_Percent        4759 non-null   float64
 7   Quantity_MT               4759 non-null   float64
 8   Price_per_MT_INR          4759 non-null   float64
 9   Total_Sales_INR           4759 non-null   float64
 10  Logistics_Cost_INR        4759 non-null   float64
 11  Profit_Margin_Percent     4759 non-null   float64
 12  Payment_Mode              4759 non-null   object
 13  Credit_Period_Days        4759 non-null   int64
 14  Bagging_Type              4759 non-null   object
dtypes: float64(7), int64(1), object(7)
memory usage: 557.8+ KB
```

Out[8]:

	Transaction_Id	Date	Customer_Type	Crop_Type	Harvest_Season	Moisture_Content_Percent	Dm_Content_Percent	Quantity_M
0	TXN2022-000077	2022-07-08	Co-operative	Maize	Kharif	69.1	30.9	9.3
1	TXN2022-000092	2022-09-28	Individual Farmer	Bajra	Kharif	67.4	32.6	8.2
2	TXN2022-000126	2022-12-06	Individual Farmer	Maize	Rabi	64.8	35.2	12.1
3	TXN2022-000128	2022-12-06	Dairy Farm	Bajra	Rabi	59.1	40.9	3.4
4	TXN2022-000133	2022-06-11	Individual Farmer	Jowar	Summer	68.4	31.6	5.6

In [9]:

```
df['Date'] = pd.to_datetime(df['Date'], format='%d-%m-%Y')
```

In [10]:

```
df.columns
```

Out[10]:

```
Index(['Transaction_Id', 'Date', 'Customer_Type', 'Crop_Type',
       'Harvest_Season', 'Moisture_Content_Percent', 'Dm_Content_Percent',
       'Quantity_Mt', 'Price_Per_Mt_Inr', 'Total_Sales_Inr',
       'Logistics_Cost_Inr', 'Profit_Margin_Percent', 'Payment_Mode',
       'Credit_Period_Days', 'Bagging_Type'],
      dtype='object')
```

In [11]:

```
df.head()
```

Out[11]:

	Transaction_Id	Date	Customer_Type	Crop_Type	Harvest_Season	Moisture_Content_Percent	Dm_Content_Percent	Quantity_M
0	TXN2022-000077	2022-07-08	Co-operative	Maize	Kharif	69.1	30.9	9.3
1	TXN2022-000092	2022-09-28	Individual Farmer	Bajra	Kharif	67.4	32.6	8.2
2	TXN2022-000126	2022-12-06	Individual Farmer	Maize	Rabi	64.8	35.2	12.1
3	TXN2022-000128	2022-12-06	Dairy Farm	Bajra	Rabi	59.1	40.9	3.4
4	TXN2022-000133	2022-06-11	Individual Farmer	Jowar	Summer	68.4	31.6	5.6

In [ ]:

4.Feature Engineering`

In [12]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4759 entries, 0 to 4758
Data columns (total 15 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Transaction_Id            4759 non-null   object
 1   Date                      4759 non-null   datetime64[ns]
 2   Customer_Type             4759 non-null   object
 3   Crop_Type                 4759 non-null   object
 4   Harvest_Season            4759 non-null   object
 5   Moisture_Content_Percent  4759 non-null   float64
 6   Dm_Content_Percent        4759 non-null   float64
 7   Quantity_Mt               4759 non-null   float64
 8   Price_Per_Mt_Inr          4759 non-null   float64
 9   Total_Sales_Inr           4759 non-null   float64
 10  Logistics_Cost_Inr        4759 non-null   float64
 11  Profit_Margin_Percent     4759 non-null   float64
 12  Payment_Mode              4759 non-null   object
 13  Credit_Period_Days        4759 non-null   int64
 14  Bagging_Type              4759 non-null   object
dtypes: datetime64[ns](1), float64(7), int64(1), object(6)
memory usage: 557.8+ KB
```

In [13]:
```
def classify_quality(row):

    moisture = row["Moisture_Content_Percent"]

    # Ideal silage ranges
    if 62 <= moisture <= 68:
        return "High Quality Silage"

    else:
        return "Average Quality Silage"


df["Silage_Quality"] = df.apply(classify_quality, axis=1)



def quantity_discount(q):
    if q < 5:
        return 0.025
    elif q < 10:
        return 0.05
    elif q < 25:
        return 0.10
    else:
        return 0.15

df["Discount_rate"] = df["Quantity_Mt"].apply(quantity_discount).astype(float)



df.columns
```

Out[13]:
```
Index(['Transaction_Id', 'Date', 'Customer_Type', 'Crop_Type',
       'Harvest_Season', 'Moisture_Content_Percent', 'Dm_Content_Percent',
       'Quantity_Mt', 'Price_Per_Mt_Inr', 'Total_Sales_Inr',
       'Logistics_Cost_Inr', 'Profit_Margin_Percent', 'Payment_Mode',
       'Credit_Period_Days', 'Bagging_Type', 'Silage_Quality',
       'Discount_rate'],
      dtype='object')
```

In [14]:
```
df.to_sql(
    name="sql_eda_silage_clean",
    con=engine,
    if_exists="replace",
    index=False,
    method="multi"
)

print("Clean dataset saved successfully")
```

```
Clean dataset saved successfully
```

In [15]:
```
%load_ext sql

%sql mysql+pymysql://root:root@localhost:3306/Pancham_Silage_Factory_DB
```

In [41]:
```
# ================================
# Global Chart Styling
# ================================

sns.set_style("whitegrid")

plt.rcParams["figure.figsize"] = (12,5)
plt.rcParams["axes.titleweight"] = "bold"
plt.rcParams["axes.labelweight"] = "bold"

def style_chart(title, xlabel, ylabel):

    plt.title(title, fontsize=14, weight="bold")

    plt.xlabel(xlabel, fontsize=11, weight="bold")

    plt.ylabel(ylabel, fontsize=11, weight="bold")

    plt.xticks(fontsize=10, weight="bold")
    plt.yticks(fontsize=10, weight="bold")

def add_labels(ax, prefix="", suffix=""):

    for p in ax.patches:
        value = p.get_height()

        ax.annotate(
            f"{prefix}{value:,.0f}{suffix}",
            (p.get_x() + p.get_width()/2., value),
            ha="center",
            va="bottom",
            fontsize=11,
            fontweight="bold",
            xytext=(0,4),
            textcoords="offset points"
        )
```

BUSINESS EDA QUESTIONS

Q1 — Which Crop Type Generates the Highest Revenue?

SQL Business Query

In [30]:
```
%%sql
SELECT
Crop_Type,
ROUND(SUM(Total_Sales_Inr),2) AS Revenue
FROM sql_eda_silage_clean
GROUP BY Crop_Type
ORDER BY Revenue DESC;
```

```
 * mysql+pymysql://root:***@localhost:3306/Pancham_Silage_Factory_DB
4 rows affected.
```

Out[30]:

Crop_Type	Revenue
Maize	148814367.34
Jowar	86479000.42
Bajra	63008685.44
Hybrid Mix	39401852.21

Python Business Query

In [36]:

```
q1 = (
df.groupby("Crop_Type")["Total_Sales_Inr"]
.sum()
.reset_index(name="Revenue")
.sort_values(by="Revenue", ascending=False)
)


q1 = q1.round({"Revenue": 2})
q1.style.format({"Total_Revenue": "{:,.2f}"})
```

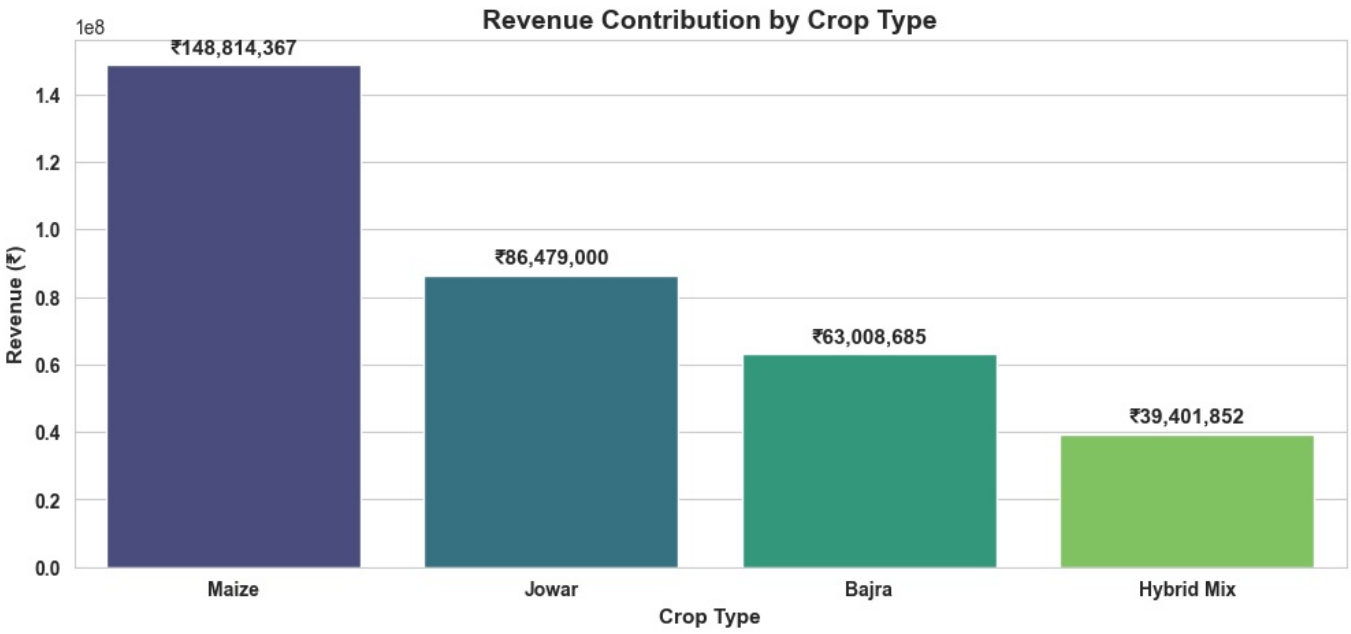
Out[36]:

	Crop_Type	Revenue
3	Maize	148814367.340000
2	Jowar	86479000.420000
0	Bajra	63008685.440000
1	Hybrid Mix	39401852.210000

Visualization of Query

In [42]:

```
ax = sns.barplot(
    data=q1,
    x="Crop_Type",
    y="Revenue",
    palette="viridis"
)

add_labels(ax, prefix="₹")

style_chart(
"Revenue Contribution by Crop Type",
"Crop Type",
"Revenue (₹)"
)

plt.show()
```



Q2 Which Customer Segment Buys the Most Quantity?

SQL Business Query

In [38]:

```
%%sql
```

```sql
SELECT
Customer_Type,
ROUND(SUM(Quantity_Mt),2) AS Total_Quantity
FROM sql_eda_silage_clean
GROUP BY Customer_Type
ORDER BY Total_Quantity DESC;
```

```
 * mysql+pymysql://root:***@localhost:3306/Pancham_Silage_Factory_DB
4 rows affected.
```

Out[38]:

Customer_Type	Total_Quantity
Individual Farmer	18012.58
Dairy Farm	16004.31
Co-operative	8886.06
Distributor	5303.75

Python Business Query

In [39]:

```python
q2 = (
df.groupby("Customer_Type")["Quantity_Mt"]
.sum()
.reset_index(name="Total_Quantity")
.sort_values(by="Total_Quantity", ascending=False)
)

q2.style.format({"Revenue": "{:,.2f}"})
```

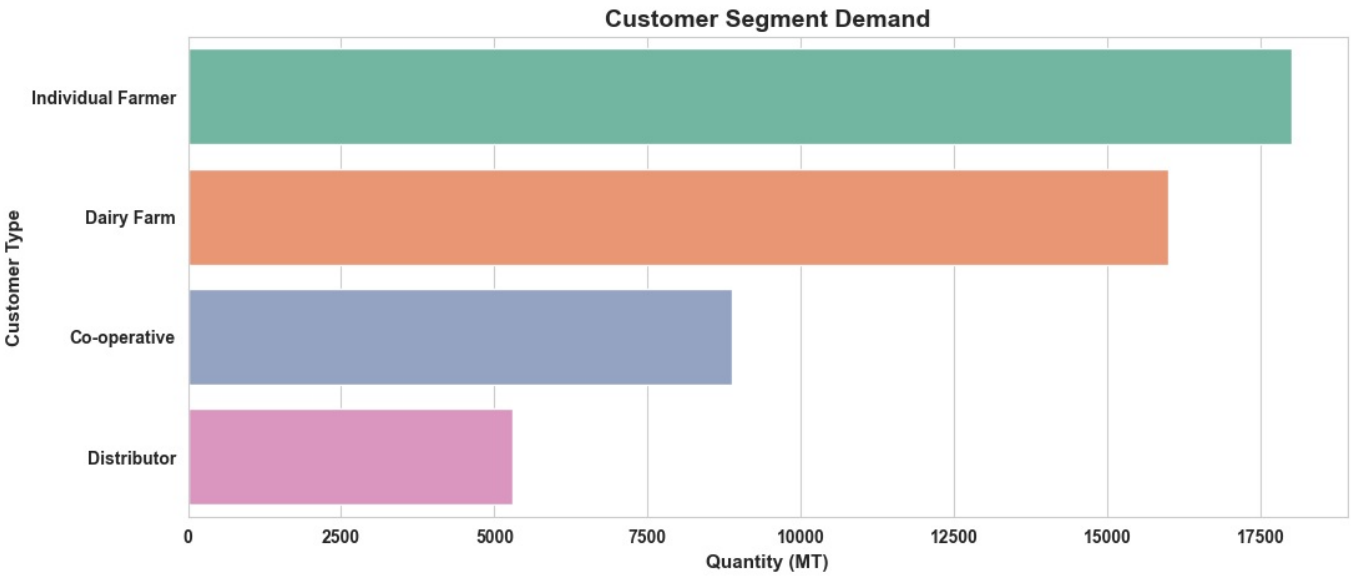
Out[39]:

	Customer_Type	Total_Quantity
3	Individual Farmer	18012.580000
1	Dairy Farm	16004.310000
0	Co-operative	8886.060000
2	Distributor	5303.750000

Visualization of Query

In [43]:

```python
ax = sns.barplot(
    data=q2,
    y="Customer_Type",
    x="Total_Quantity",
    palette="Set2"
)

style_chart(
"Customer Segment Demand",
"Quantity (MT)",
"Customer Type"
)

plt.show()
```



Q3 — How does silage price vary across harvest seasons with different types of crops?

SQL Business Query

In [46]:
```
%%sql
SELECT
Harvest_Season,
Crop_Type,
ROUND(AVG(Price_Per_Mt_Inr),2) AS Avg_Price
FROM sql_eda_silage_clean
GROUP BY Harvest_Season, Crop_Type
ORDER BY Harvest_Season, Avg_Price ASC;
```

```
 * mysql+pymysql://root:***@localhost:3306/Pancham_Silage_Factory_DB
12 rows affected.
```

Out[46]:

Harvest_Season	Crop_Type	Avg_Price
Kharif	Hybrid Mix	6246.05
Kharif	Maize	6777.25
Kharif	Bajra	7159.77
Kharif	Jowar	8097.06
Rabi	Hybrid Mix	6202.16
Rabi	Maize	6734.89
Rabi	Bajra	7122.92
Rabi	Jowar	8036.79
Summer	Hybrid Mix	6562.94
Summer	Maize	7129.43
Summer	Bajra	7494.32
Summer	Jowar	8398.42

Python Business Query

In [50]:
```
q3 = (
df.groupby(["Harvest_Season","Crop_Type"])["Price_Per_Mt_Inr"]
.mean()
.reset_index(name="Avg_Price")
)

q3.round({"Avg_Price": 2})
q3.style.format({"Avg_Price": "₹{:,.2f}"})
```

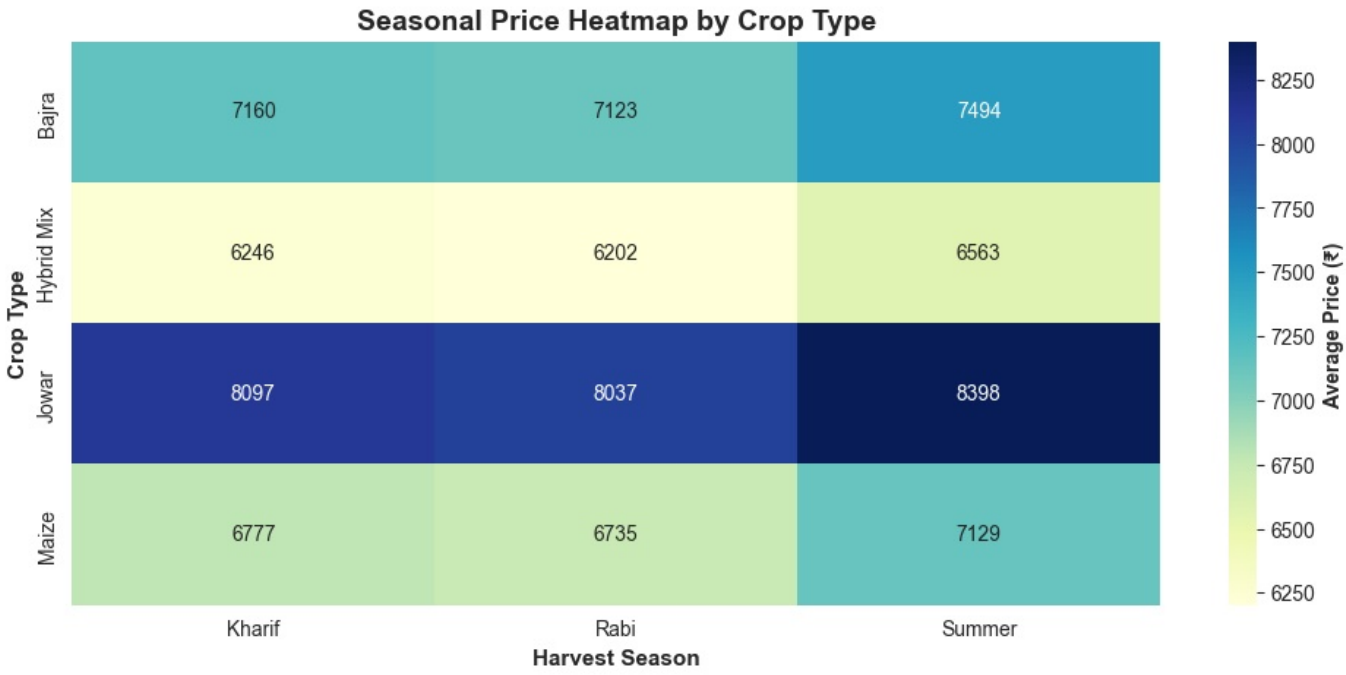
Out[50]:

	Harvest_Season	Crop_Type	Avg_Price
0	Kharif	Bajra	₹7,159.77
1	Kharif	Hybrid Mix	₹6,246.05
2	Kharif	Jowar	₹8,097.06
3	Kharif	Maize	₹6,777.25
4	Rabi	Bajra	₹7,122.92
5	Rabi	Hybrid Mix	₹6,202.16
6	Rabi	Jowar	₹8,036.79
7	Rabi	Maize	₹6,734.89
8	Summer	Bajra	₹7,494.32
9	Summer	Hybrid Mix	₹6,562.94
10	Summer	Jowar	₹8,398.42
11	Summer	Maize	₹7,129.43

Visualization of Query

In [60]:
```
pivot_table = q3.pivot(
    index="Crop_Type",
    columns="Harvest_Season",
    values="Avg_Price"
)

plt.figure(figsize=(12,5))

sns.heatmap(
    pivot_table,
    annot=True,
    fmt=".0f",
    cmap="YlGnBu",
```

```
    cbar_kws={"label": "Average Price (₹)"}
)

plt.title("Seasonal Price Heatmap by Crop Type",fontsize=14,weight="bold")

plt.xlabel("Harvest Season",fontsize=11,weight="bold")

plt.ylabel("Crop Type",fontsize=11,weight="bold")

plt.show()
```



Q4 -- Packaging Wise Profit vs Price per Ton

SQL Business Query

In [70]:
```
%%sql
SELECT
Bagging_Type,
COUNT(*) AS Total_Orders,
ROUND(SUM(Quantity_Mt),2) AS Total_Quantity_MT
FROM sql_eda_silage_clean
GROUP BY Bagging_Type
ORDER BY Total_Orders DESC;
```

```
 * mysql+pymysql://root:***@localhost:3306/Pancham_Silage_Factory_DB
4 rows affected.
```

Out[70]:

Bagging_Type	Total_Orders	Total_Quantity_MT
Bale-50kg	1816	18232.45
Bulk	1301	13301.09
Bags	1288	13208.86
Bale-25kg	354	3464.3

Python Business Query

In [72]:
```
q4 = (
df.groupby("Bagging_Type")
.agg(
Total_Orders=("Bagging_Type","count"),
Total_Quantity_MT=("Quantity_Mt","sum")
)
.reset_index()
.sort_values(by="Total_Orders",ascending=False)
)

q4
```

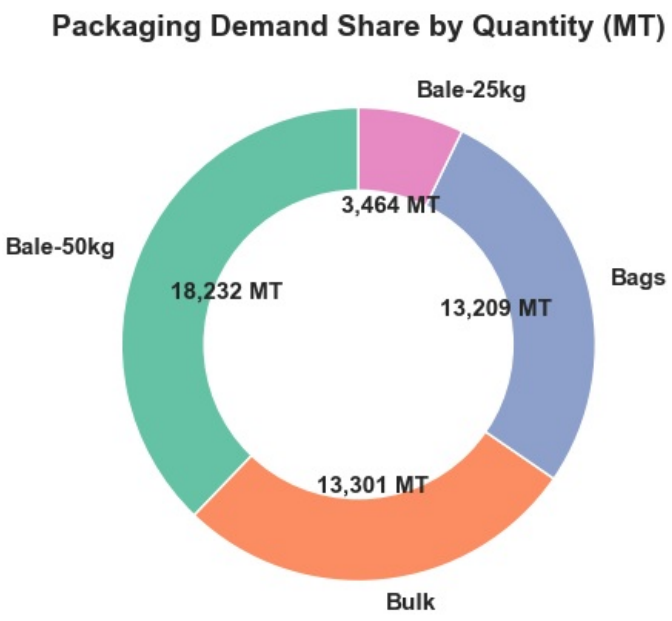
Out[72]:

	Bagging_Type	Total_Orders	Total_Quantity_MT
2	Bale-50kg	1816	18232.45
3	Bulk	1301	13301.09
0	Bags	1288	13208.86
1	Bale-25kg	354	3464.30

Visualization of Query

In [79]:

```
plt.figure(figsize=(12,5))

colors = sns.color_palette("Set2")

wedges, texts, autotexts = plt.pie(
    q4["Total_Quantity_MT"],
    labels=q4["Bagging_Type"],
    autopct=lambda p: f"{(p/100)*q4['Total_Quantity_MT'].sum():,.0f} MT",
    colors=colors,
    startangle=90,
    textprops={"fontsize":11, "fontweight":"bold"}
)

# make quantity text bold
for autotext in autotexts:
    autotext.set_fontsize(11)
    autotext.set_fontweight("bold")

# Donut hole
centre_circle = plt.Circle((0,0),0.65,fc='white')
fig = plt.gcf()
fig.gca().add_artist(centre_circle)

plt.title(
    "Packaging Demand Share by Quantity (MT)",
    fontsize=14,
    weight="bold"
)

plt.show()
```



Q5 — Which payment methods are most commonly used?

SQL Business Query

In [29]:

```
%%sql
SELECT
Payment_Mode,
COUNT(*) AS Transactions
FROM sql_eda_silage_clean
GROUP BY Payment_Mode
ORDER BY Transactions DESC;
```

```
 * mysql+pymysql://root:***@localhost:3306/Pancham_Silage_Factory_DB
4 rows affected.
```

Out[29]:

Payment_Mode	Transactions
UPI	1879
Bank Transfer	1465
Credit	935
Cash	480

Python Business Query

In [30]:

```
q5 = df["Payment_Mode"].value_counts().reset_index()

q5.columns = ["Payment_Mode","Transactions"]

q5
```

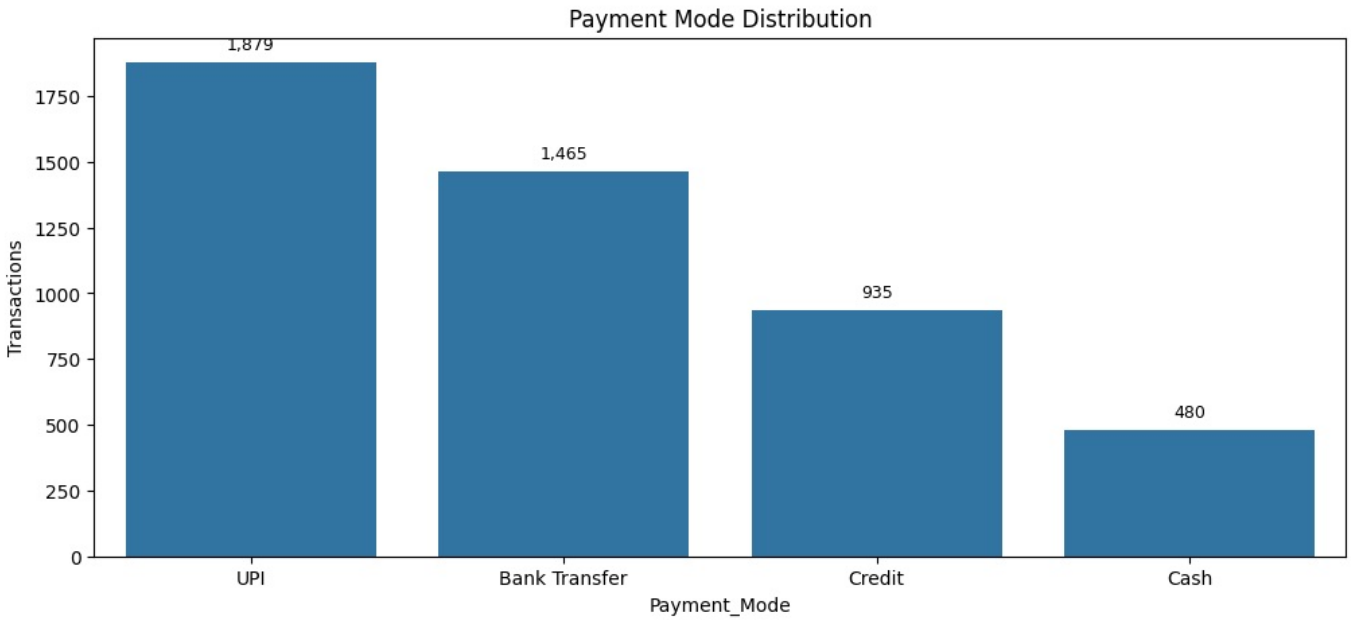
Out[30]:

	Payment_Mode	Transactions
0	UPI	1879
1	Bank Transfer	1465
2	Credit	935
3	Cash	480

Visualization of Query

In [31]:

```
plt.figure(figsize=(12,5))

ax = sns.barplot(data=q5,x="Payment_Mode",y="Transactions")

add_labels(ax)

plt.title("Payment Mode Distribution")

plt.show()
```



Q6 — Does bulk purchasing reduce the price per metric ton?

SQL Business Query

In [82]:

```
%%sql
SELECT
Crop_Type,

CASE
    WHEN Quantity_Mt < 5 THEN 'Below 5 MT'
    WHEN Quantity_Mt BETWEEN 5 AND 10 THEN '5 - 10 MT'
    WHEN Quantity_Mt BETWEEN 10 AND 25 THEN '10 - 25 MT'
    ELSE 'Above 25 MT'
END AS Bulk_Category,
```

```sql
ROUND(AVG(Price_Per_Mt_Inr),2) AS Avg_Price

FROM sql_eda_silage_clean

GROUP BY Crop_Type, Bulk_Category

ORDER BY Crop_Type, Avg_Price;
```

```
 * mysql+pymysql://root:***@localhost:3306/Pancham_Silage_Factory_DB
16 rows affected.
```

Out[82]:

Crop_Type	Bulk_Category	Avg_Price
Bajra	10 - 25 MT	7223.52
Bajra	5 - 10 MT	7301.74
Bajra	Below 5 MT	7336.27
Bajra	Above 25 MT	7402.78
Hybrid Mix	10 - 25 MT	6288.89
Hybrid Mix	5 - 10 MT	6401.06
Hybrid Mix	Below 5 MT	6439.31
Hybrid Mix	Above 25 MT	6522.25
Jowar	10 - 25 MT	8118.69
Jowar	Above 25 MT	8166.04
Jowar	5 - 10 MT	8238.06
Jowar	Below 5 MT	8292.21
Maize	10 - 25 MT	6837.72
Maize	5 - 10 MT	6890.82
Maize	Below 5 MT	6961.59
Maize	Above 25 MT	7069.08

Python Business Query

In [84]:

```python
df["Bulk_Category"] = pd.cut(
    df["Quantity_Mt"],
    bins=[0,5,10,25,100],
    labels=["Below 5 MT","5-10 MT","10-25 MT","Above 25 MT"]
)

q_bulk = (
    df.groupby(["Crop_Type","Bulk_Category"])["Price_Per_Mt_Inr"]
    .mean()
    .reset_index(name="Avg_Price")
)

q_bulk.round({"Avg_Price": 2})
q_bulk.style.format({"Avg_Price": "₹{:,.2f}"})
```

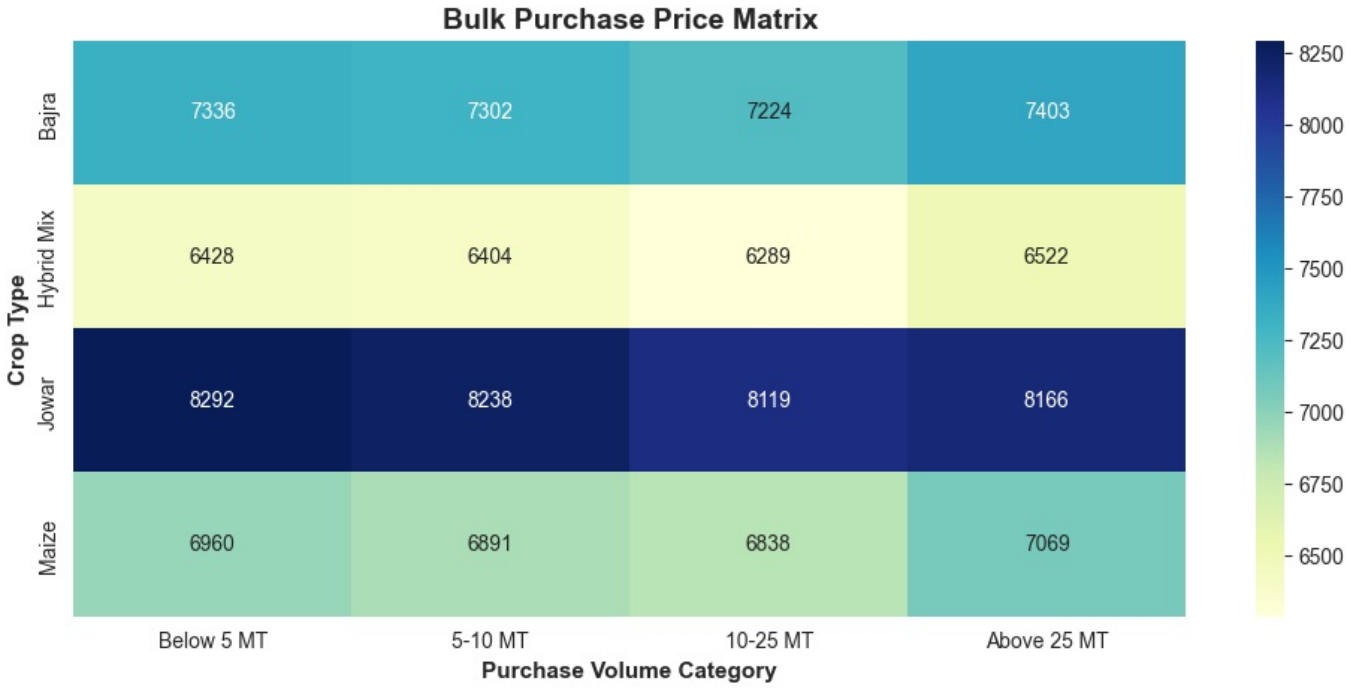
Out[84]:

	Crop_Type	Bulk_Category	Avg_Price
0	Bajra	Below 5 MT	₹7,336.27
1	Bajra	5-10 MT	₹7,301.74
2	Bajra	10-25 MT	₹7,223.52
3	Bajra	Above 25 MT	₹7,402.78
4	Hybrid Mix	Below 5 MT	₹6,428.39
5	Hybrid Mix	5-10 MT	₹6,403.57
6	Hybrid Mix	10-25 MT	₹6,288.89
7	Hybrid Mix	Above 25 MT	₹6,522.25
8	Jowar	Below 5 MT	₹8,292.21
9	Jowar	5-10 MT	₹8,238.06
10	Jowar	10-25 MT	₹8,118.69
11	Jowar	Above 25 MT	₹8,166.04
12	Maize	Below 5 MT	₹6,959.55
13	Maize	5-10 MT	₹6,891.27
14	Maize	10-25 MT	₹6,837.72
15	Maize	Above 25 MT	₹7,069.08

Visualization of Query

In [85]:

```
pivot_bulk = q_bulk.pivot(
    index="Crop_Type",
    columns="Bulk_Category",
    values="Avg_Price"
)

plt.figure(figsize=(12,5))

sns.heatmap(
    pivot_bulk,
    annot=True,
    cmap="YlGnBu",
    fmt=".0f"
)

plt.title(
"Bulk Purchase Price Matrix",
fontsize=14,
weight="bold"
)

plt.xlabel(
"Purchase Volume Category",
fontsize=11,
weight="bold"
)

plt.ylabel(
"Crop Type",
fontsize=11,
weight="bold"
)

plt.show()
```



Q7 — Which crop gives the highest profit margin?

SQL Business Query

In [86]:
```
%%sql
SELECT
Crop_Type,
ROUND(AVG(Profit_Margin_Percent),2) AS Avg_Profit
FROM sql_eda_silage_clean
GROUP BY Crop_Type
ORDER BY Avg_Profit DESC;
```

```
 * mysql+pymysql://root:***@localhost:3306/Pancham_Silage_Factory_DB
4 rows affected.
```

Out[86]:

Crop_Type	Avg_Profit
Hybrid Mix	14.58
Bajra	14.57
Jowar	14.53
Maize	14.48

Python Business Query

In [88]:
```
q7 = (
df.groupby("Crop_Type")["Profit_Margin_Percent"]
.mean()
.reset_index(name="Avg_Profit")
)

q7.round({"Avg_Profit": 2})
q7.style.format({"Avg_Profit": "{:.2f}%"})
```

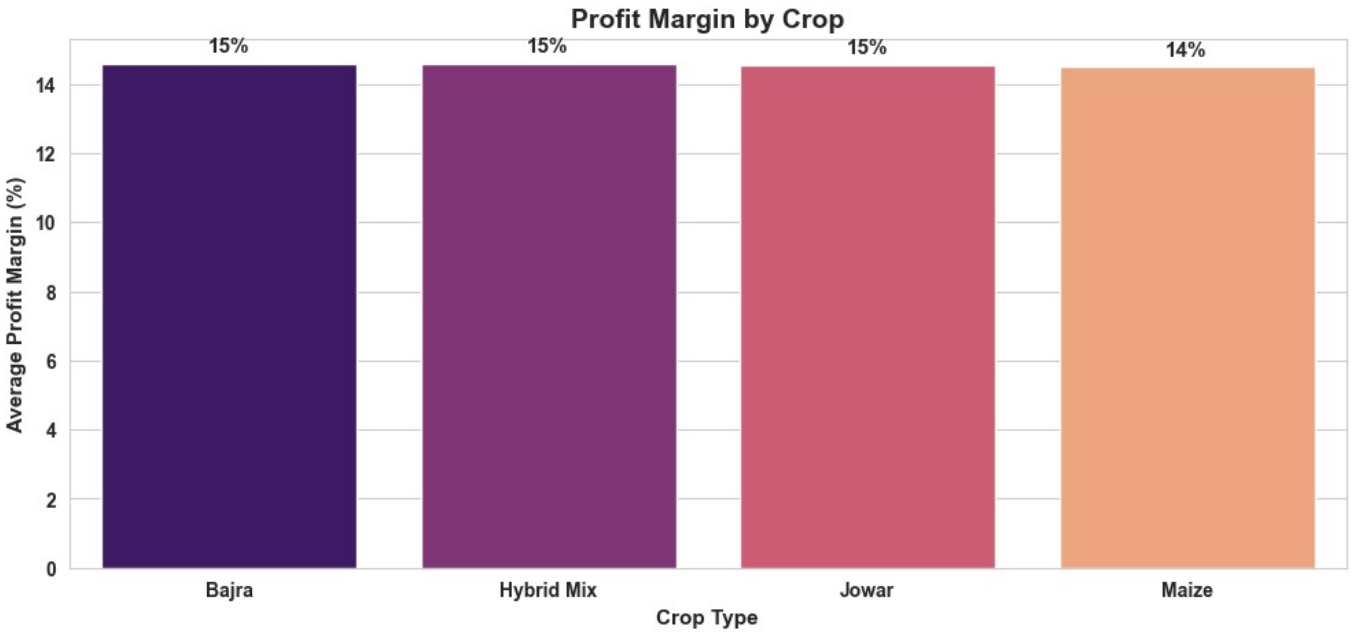
Out[88]:

	Crop_Type	Avg_Profit
0	Bajra	14.57%
1	Hybrid Mix	14.58%
2	Jowar	14.53%
3	Maize	14.48%

Visualization of Query

In [89]:
```
ax = sns.barplot(
data=q7,
x="Crop_Type",
y="Avg_Profit",
palette="magma"
)

add_labels(ax,suffix="%")
```

```
style_chart(
"Profit Margin by Crop",
"Crop Type",
"Average Profit Margin (%)"
)

plt.show()
```



Q8 — Which customer segment takes the longest credit period?

SQL Business Query

In [91]:
```
%%sql
SELECT
Customer_Type,
ROUND(AVG(Credit_Period_Days),2) AS Avg_Credit
FROM sql_eda_silage_clean
GROUP BY Customer_Type
ORDER BY Avg_Credit DESC;
```

```
 * mysql+pymysql://root:***@localhost:3306/Pancham_Silage_Factory_DB
4 rows affected.
```

Out[91]:

Customer_Type	Avg_Credit
Individual Farmer	4.35
Distributor	3.80
Dairy Farm	3.72
Co-operative	3.45

Python Business Query

In [94]:
```
q8 = (
df.groupby("Customer_Type")["Credit_Period_Days"]
.mean()
.reset_index(name="Avg_Credit")
.sort_values(by="Avg_Credit",ascending=False)
)

q8.Avg_Credit.round(2)
q8.style.format({"Avg_Credit": "{:.2f} days"})
```

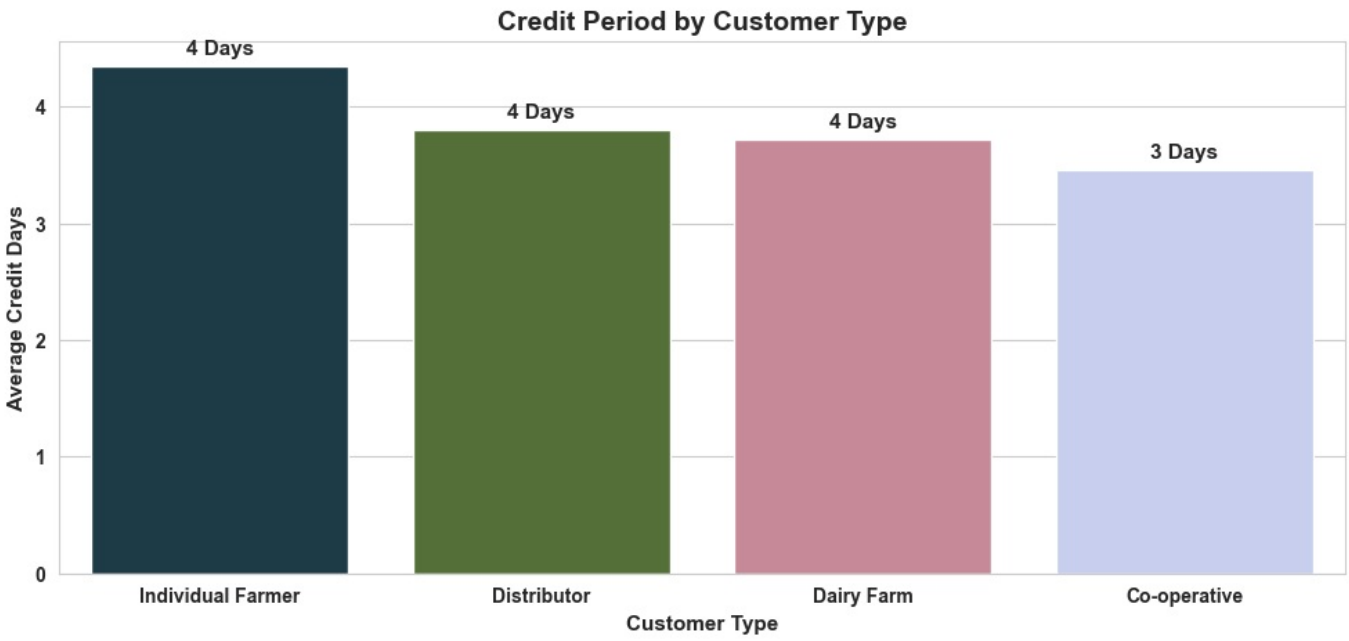
Out[94]:

	Customer_Type	Avg_Credit
3	Individual Farmer	4.35 days
2	Distributor	3.80 days
1	Dairy Farm	3.72 days
0	Co-operative	3.45 days

Visualization of Query

In [95]:
```
ax = sns.barplot(
```

```
data=q8,
x="Customer_Type",
y="Avg_Credit",
palette="cubehelix"
)

add_labels(ax,suffix=" Days")

style_chart(
"Credit Period by Customer Type",
"Customer Type",
"Average Credit Days"
)

plt.show()
```



Q9 — How do silage sales change over time?

SQL Business Query

In [96]:
```
%%sql
SELECT
CONCAT(YEAR(Date),' Q',QUARTER(Date)) AS Quarter,
ROUND(AVG(Total_Sales_Inr),2) AS Avg_Sales
FROM sql_eda_silage_clean
GROUP BY Quarter
ORDER BY Quarter;
```

```
 * mysql+pymysql://root:***@localhost:3306/Pancham_Silage_Factory_DB
15 rows affected.
```

Out[96]:

Quarter	Avg_Sales
2022 Q1	74638.15
2022 Q2	71996.21
2022 Q3	73655.77
2022 Q4	69029.44
2023 Q1	68660.63
2023 Q2	75771.24
2023 Q3	70760.49
2023 Q4	72301.74
2024 Q1	69136.04
2024 Q2	75534.82
2024 Q3	67862.94
2024 Q4	68817.4
2025 Q1	71235.54
2025 Q2	72827.6
2025 Q3	67900.0

Python Business Query

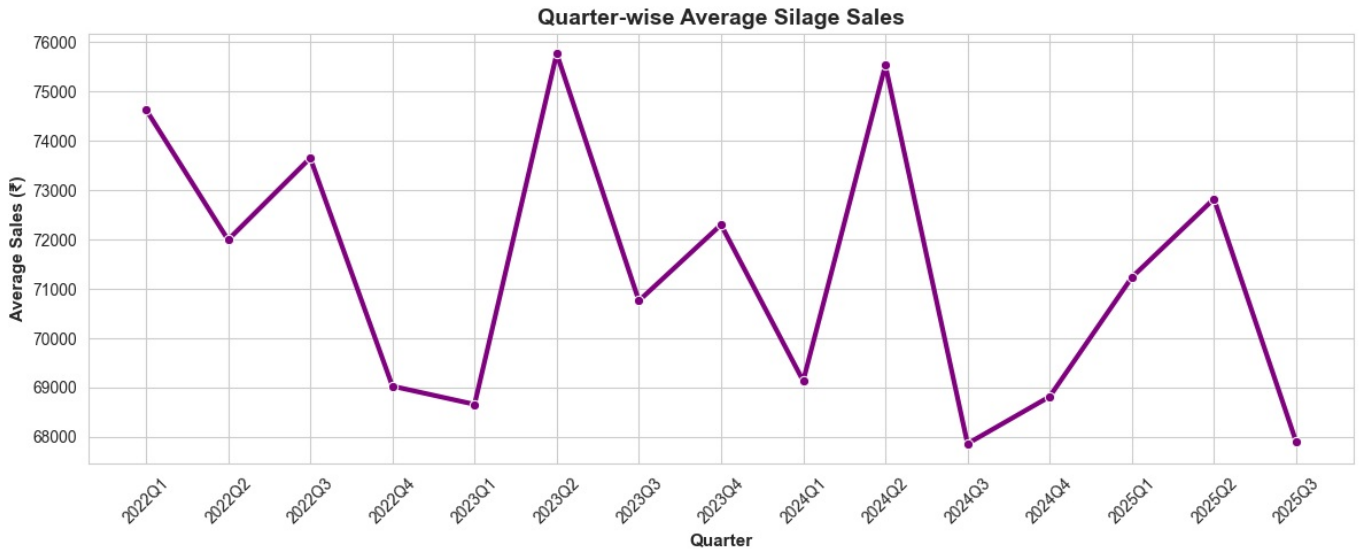
In [97]:
```
df["Quarter"] = df["Date"].dt.to_period("Q").astype(str)

q_quarter = (
    df.groupby("Quarter")["Total_Sales_Inr"]
    .mean()
    .reset_index(name="Avg_Sales")
)

q_quarter
```

Out[97]:

	Quarter	Avg_Sales
0	2022Q1	74638.152500
1	2022Q2	71996.207143
2	2022Q3	73655.772212
3	2022Q4	69029.436977
4	2023Q1	68660.625602
5	2023Q2	75771.238907
6	2023Q3	70760.485868
7	2023Q4	72301.739048
8	2024Q1	69136.042751
9	2024Q2	75534.821192
10	2024Q3	67862.939898
11	2024Q4	68817.396234
12	2025Q1	71235.536654
13	2025Q2	72827.604660
14	2025Q3	67900.001879

Visualization of Query

In [100…
```
plt.figure(figsize=(12,5))

sns.lineplot(
    data=q_quarter,
    x="Quarter",
    y="Avg_Sales",
    marker="o",
    linewidth=3,
    color="purple"
)

plt.title(
"Quarter-wise Average Silage Sales",
fontsize=14,
weight="bold"
)

plt.xlabel(
"Quarter",
fontsize=11,
weight="bold"
)

plt.ylabel(
"Average Sales (₹)",
fontsize=11,
weight="bold"
)

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()
```



Q10 — Does moisture content affect silage pricing?

SQL Business Query

In [51]:
```
%%sql
SELECT Moisture_Content_Percent, Price_Per_Mt_Inr
FROM sql_eda_silage_clean
LIMIT 10;
```

```
 * mysql+pymysql://root:***@localhost:3306/Pancham_Silage_Factory_DB
10 rows affected.
```

Out[51]:

Moisture_Content_Percent	Price_Per_Mt_Inr
69.1	6284.0
67.4	7608.0
64.8	6885.0
59.1	6664.0
68.4	8161.0
67.5	6867.0
63.9	6469.0
62.7	8605.0
60.5	6646.0
63.9	8745.0

Python Business Query

In [102…
```
q_moisture = df[["Moisture_Content_Percent","Price_Per_Mt_Inr"]]

q_moisture.head()
```

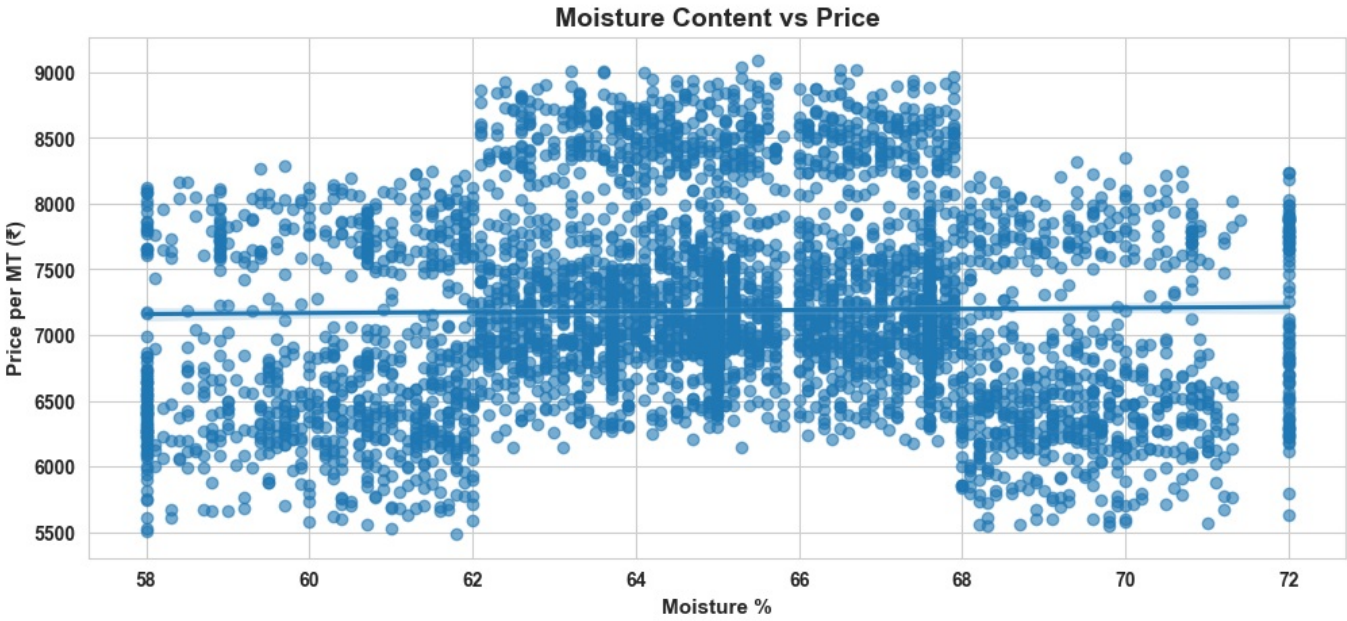
Out[102…

	Moisture_Content_Percent	Price_Per_Mt_Inr
0	69.1	6284.0
1	67.4	7608.0
2	64.8	6885.0
3	59.1	6664.0
4	68.4	8161.0

Visualization of Query

In [101…
```
sns.regplot(
data=df,
x="Moisture_Content_Percent",
y="Price_Per_Mt_Inr",
scatter_kws={"alpha":0.6}
)

style_chart(
"Moisture Content vs Price",
```

```
"Moisture %",
"Price per MT (₹)"
)

plt.show()
```



Q11 — Which crops are most demanded in each season?

SQL Business Query

In [103…

```
%%sql
SELECT
Harvest_Season,
Crop_Type,
COUNT(*) AS Orders
FROM sql_eda_silage_clean
GROUP BY Harvest_Season, Crop_Type;
```

```
 * mysql+pymysql://root:***@localhost:3306/Pancham_Silage_Factory_DB
12 rows affected.
```

Out[103…

Harvest_Season	Crop_Type	Orders
Kharif	Maize	941
Kharif	Bajra	151
Rabi	Maize	527
Rabi	Bajra	380
Summer	Jowar	425
Rabi	Hybrid Mix	215
Kharif	Jowar	302
Summer	Bajra	344
Rabi	Jowar	364
Summer	Maize	703
Kharif	Hybrid Mix	149
Summer	Hybrid Mix	258

Python Business Query

In [104…

```
q11 = (
df.groupby(["Harvest_Season","Crop_Type"])
.size()
.reset_index(name="Orders")
)

q11
```

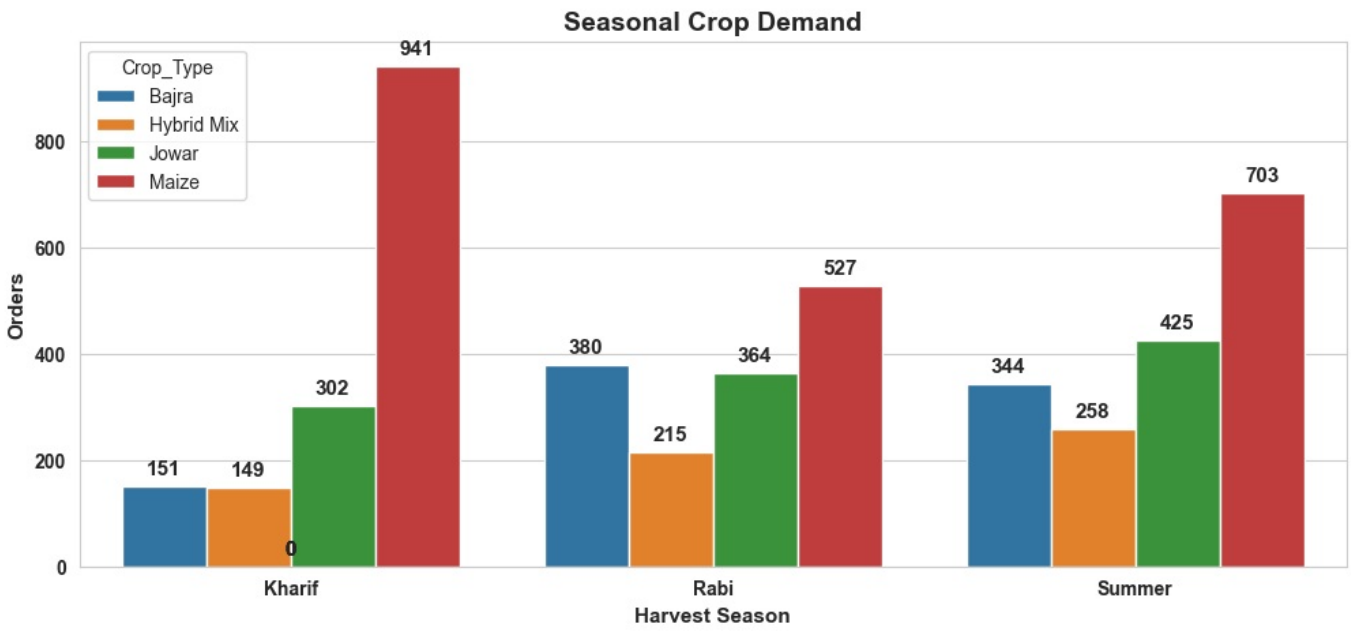
Out[104…

	Harvest_Season	Crop_Type	Orders
0	Kharif	Bajra	151
1	Kharif	Hybrid Mix	149
2	Kharif	Jowar	302
3	Kharif	Maize	941
4	Rabi	Bajra	380
5	Rabi	Hybrid Mix	215
6	Rabi	Jowar	364
7	Rabi	Maize	527
8	Summer	Bajra	344
9	Summer	Hybrid Mix	258
10	Summer	Jowar	425
11	Summer	Maize	703

Visualization of Query

In [106…

```
sns.barplot(
data=q11,
x="Harvest_Season",
y="Orders",
hue="Crop_Type",
palette="tab10"
)

style_chart(
"Seasonal Crop Demand",
"Harvest Season",
"Orders"
)

add_labels(plt.gca())

plt.show()
```



Q12 — How do Total Sales change Month-wise?

SQL Business Query

In [111…

```
%%sql
SELECT
DATE_FORMAT(Date,'%Y-%m') AS Month,
ROUND(SUM(Total_Sales_Inr),2) AS Total_Sales
FROM sql_eda_silage_clean
GROUP BY Month
ORDER BY Month;
```

```
 * mysql+pymysql://root:***@localhost:3306/Pancham_Silage_Factory_DB
44 rows affected.
```

Out[111…

Month	Total_Sales
2022-01	31592.88
2022-02	625163.11
2022-03	1134559.67
2022-04	1021058.26
2022-05	2197175.27
2022-06	1821500.97
2022-07	2144620.85
2022-08	2293534.58
2022-09	3222044.88
2022-10	3094717.54
2022-11	3966584.29
2022-12	4811761.33
2023-01	3821630.73
2023-02	4852143.96
2023-03	4440404.8
2023-04	5971050.27
2023-05	6737314.31
2023-06	6007131.43
2023-07	6997599.27
2023-08	7021525.81
2023-09	6359894.85
2023-10	7491962.74
2023-11	7484231.46
2023-12	9317190.12
2024-01	8754196.14
2024-02	8831008.59
2024-03	9308715.9
2024-04	11099022.33
2024-05	10947019.85
2024-06	10282861.29
2024-07	10636782.46
2024-08	11120200.78
2024-09	11360131.43
2024-10	11448831.84
2024-11	12673010.59
2024-12	13521273.31
2025-01	13549882.3
2025-02	12050928.75
2025-03	12937614.28
2025-04	14165613.52
2025-05	13934114.1
2025-06	14722903.92
2025-07	15050313.71
2025-08	8443086.94

Python Business Query

In [112…

```
df["Month"] = df["Date"].dt.to_period("M").astype(str)

q_sales_month = (
    df.groupby("Month")["Total_Sales_Inr"]
    .sum()
    .reset_index(name="Total_Sales")
```

```
    .sort_values(by="Month")
)

q_sales_month
```

Out[112…

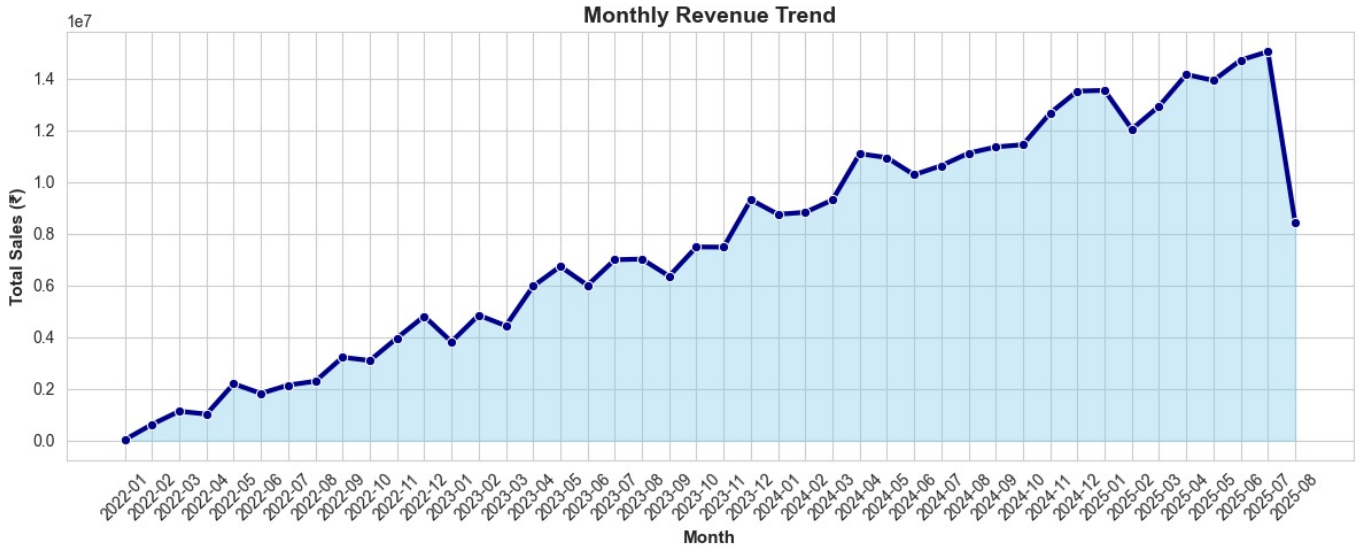
	Month	Total_Sales
0	2022-01	31592.88
1	2022-02	625163.11
2	2022-03	1134559.67
3	2022-04	1021058.26
4	2022-05	2197175.27
5	2022-06	1821500.97
6	2022-07	2144620.85
7	2022-08	2293534.58
8	2022-09	3222044.88
9	2022-10	3094717.54
10	2022-11	3966584.29
11	2022-12	4811761.33
12	2023-01	3821630.73
13	2023-02	4852143.96
14	2023-03	4440404.80
15	2023-04	5971050.27
16	2023-05	6737314.31
17	2023-06	6007131.43
18	2023-07	6997599.27
19	2023-08	7021525.81
20	2023-09	6359894.85
21	2023-10	7491962.74
22	2023-11	7484231.46
23	2023-12	9317190.12
24	2024-01	8754196.14
25	2024-02	8831008.59
26	2024-03	9308715.90
27	2024-04	11099022.33
28	2024-05	10947019.85
29	2024-06	10282861.29
30	2024-07	10636782.46
31	2024-08	11120200.78
32	2024-09	11360131.43
33	2024-10	11448831.84
34	2024-11	12673010.59
35	2024-12	13521273.31
36	2025-01	13549882.30
37	2025-02	12050928.75
38	2025-03	12937614.28
39	2025-04	14165613.52
40	2025-05	13934114.10
41	2025-06	14722903.92
42	2025-07	15050313.71
43	2025-08	8443086.94

Visualization of Query

In [113…

```
plt.figure(figsize=(12,5))
```

```python
plt.fill_between(
    q_sales_month["Month"],
    q_sales_month["Total_Sales"],
    color="skyblue",
    alpha=0.4
)

sns.lineplot(
    data=q_sales_month,
    x="Month",
    y="Total_Sales",
    color="darkblue",
    linewidth=3,
    marker="o"
)

plt.title("Monthly Revenue Trend", fontsize=14, weight="bold")

plt.xlabel("Month", fontsize=11, weight="bold")
plt.ylabel("Total Sales (₹)", fontsize=11, weight="bold")

plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```



In [ ]: